Nombre: Cesar Maximiliano Peña Mendoza Diego Alberto Sierra Mendoza Arturo Sebastian Pérez Zapata	**Matrícula:** 2144135 2143733 2142942
Nombre del curso: COMPONENTES DE LOGICA DE NEGOCIOS	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Caso: 07 · NovaRent	**Actividad:** PIA. PROYECTO INTEGRADOR DE APRENDIZAJE (REPORTE FINAL)
Fecha: 06/11/2026	

Bibliografía (Formato APA)

Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
Ramírez, S. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com
SQLAlchemy. (s. f.). SQLAlchemy documentation. https://docs.sqlalchemy.org
pytest-dev. (s. f.). pytest documentation. https://docs.pytest.org
Astral. (s. f.). Ruff documentation. https://docs.astral.sh/ruff/
Docker. (s. f.). Docker Compose documentation. https://docs.docker.com/compose/
GitHub. (s. f.). GitHub Actions documentation. https://docs.github.com/actions
Render. (s. f.). Render documentation: Blueprints. https://render.com/docs/infrastructure-as-code

Objetivo:

Modelar e implementar la lógica de negocio del caso **NovaRent** (renta de automóviles por día) con diseño orientado al dominio (DDD), exponerla mediante una API REST documentada con OpenAPI, respaldarla con pruebas automatizadas y cobertura, y entregarla con contenedores, integración continua en GitHub Actions y despliegue público en Render.

Procedimiento:

El proyecto se organiza en capas (dominio, aplicación, infraestructura y API), con persistencia en PostgreSQL o SQLite mediante los patrones Repository y Unit of Work. Se verificó con pruebas automatizadas, se integró con un pipeline de CI que ejecuta ruff, pytest y Docker Compose, y se desplegó con un Blueprint de Render. A continuación se presentan los datos del proyecto, el resumen de resultados, el diseño, la calidad, el despliegue, las evidencias y el registro de uso de IA.

1. Introducción y resumen de resultados

NovaRent es una empresa de renta de automóviles por día. El alcance funcional incluye reservación, entrega con checklist y evidencia, contrato, extensiones, devolución, cargos automáticos (kilometraje, combustible, retraso y daños), disputas, depósito de garantía, limpieza y reportes.

1.1 Tabla de datos del proyecto

Dato	Valor
Repositorio (GitHub)	https://github.com/cesarmax90/pia_cln_proyecto_integrador_ad2026_EQ7
Aplicación desplegada	https://novarent-api-u3e9.onrender.com
Documentación OpenAPI	https://novarent-api-u3e9.onrender.com/docs
Verificación de salud	https://novarent-api-u3e9.onrender.com/salud
CI/CD	GitHub Actions · workflow «CI NovaRent» (`.github/workflows/ci.yml`)
Tecnologías	Python 3.12, FastAPI, SQLAlchemy, PostgreSQL 16 / SQLite, Docker y Docker Compose, pytest, ruff
Despliegue	Render (Blueprint `render.yaml`): servicio web **novarent-api** y base de datos **novarent-db**

Nota: la aplicación se despliega en el plan gratuito de Render; tras un periodo de inactividad la primera solicitud puede tardar 50 segundos o más.

1.2 Resumen de resultados

Verificación	Resultado	Evidencia
Pruebas automatizadas	354 aprobadas con SQLite y 356 con PostgreSQL (reportadas en el repositorio); el CI las ejecuta en Python 3.11 y 3.12	Figura 1
Cobertura	98.5 % de líneas y ramas (umbral exigido: 80 %)	README, CI
`ruff check src tests scripts`	Sin errores; job Lint en verde	Figura 1
Paquete de datos del PIA	120 casos y 20 transiciones ejecutados, 0 diferencias contra el oráculo	docs/evidencias
Pipeline en GitHub Actions	Ejecución #1 verde; #2 a #4 rojas (Docker); #5 y #6 verdes tras la corrección	Figuras 1 a 3
Despliegue en Render	novarent-api en estado Live (commit e75514a) y novarent-db creada	Figuras 4 y 5
API en producción	`/salud` correcto; `/docs`; 401 sin credenciales y 200 con `AGE-01` / `AGENTE`	Figuras 6 a 9

2. Requerimientos y documentación

La documentación se encuentra en la carpeta `docs/` del repositorio y cubre lo solicitado por el PIA:

Entregable	Ubicación
Documento de requerimientos de negocio (BRD)	`docs/BRD.md`
20 historias de usuario con criterios Gherkin (mínimo solicitado: 15)	`docs/historias_usuario.md`

Entregable	Ubicación
Modelo de procesos BPMN	`docs/bpmn.md`
Glosario del dominio	`docs/glosario.md`
Trazabilidad BRD → regla → componente → prueba	`docs/trazabilidad.md`
Adaptación y uso del paquete de datos	`docs/adaptacion_paquete_datos.md`, `docs/evidencias/*.csv`
Guion de demostración para la defensa	`docs/guion_demo.md`

3. Diseño y arquitectura

El sistema se organiza en capas con dependencias hacia el dominio:

- **Dominio** (`src/domain`): agregados, objetos de valor (`Dinero`, `PeriodoRenta`), 11 especificaciones combinables con AND/OR/NOT, 5 máquinas de estado, políticas y 19 eventos de dominio. No depende de frameworks.
- **Aplicación** (`src/application`): casos de uso, permisos por rol, DTO y manejadores de eventos.
- **Infraestructura** (`src/infrastructure`): SQLAlchemy, Repository y Unit of Work, adaptadores (pasarela de pago, evidencia) y datos demo.
- **API** (`src/api`): rutas FastAPI, esquemas y manejo de errores con los códigos 200, 201, 400, 404, 409 y 422.

Las decisiones de diseño están registradas en 10 ADR (Architecture Decision Records) dentro de `docs/architecture/`, junto con diagramas Mermaid de secuencia, estados y entidad-relación.

4. Pruebas y calidad

Verificación	Resultado reportado en el repositorio
`ruff check src tests scripts`	Sin errores
`pytest` con SQLite	354 pruebas aprobadas · cobertura de 98.5 % (mínimo: 80 %)
`pytest` con PostgreSQL 16	356 pruebas aprobadas · cobertura de 98.5 %
Paquete de datos del PIA	120 casos y 20 transiciones, 0 diferencias contra el oráculo independiente
Integración continua	Jobs de Lint, Pruebas (Python 3.11 y 3.12) y Docker Compose en verde (Figura 1)

Las pruebas cubren dominio, infraestructura, servicios de aplicación y API. De las 354 pruebas, 134 provienen del paquete de datos (120 casos parametrizados y 14 más) y 220 son de dominio, servicios y API.

5. Integración continua y despliegue

CI/CD. El workflow «CI NovaRent» se ejecuta en cada push y pull request: (1) lint con ruff, (2) pruebas con cobertura en una matriz de Python 3.11 y 3.12 contra PostgreSQL, y (3) construcción con Docker Compose, ejecución de las pruebas dentro de contenedores y verificación del endpoint `/salud`.

Despliegue. El archivo `render.yaml` define un Blueprint de Render con el servicio web **novarent-api** (Docker, plan gratuito) y la base de datos PostgreSQL **novarent-db**. El servicio quedó en estado *Live* con el commit `e75514a` de la rama `main`.

Incidente resuelto durante la entrega. Las primeras ejecuciones del CI fallaban en el paso «Pruebas dentro de contenedores» porque Docker no podía copiar la carpeta `docs` al construir la imagen de pruebas (`COPY docs ./docs: "/docs": not found`). La causa era que el archivo `.dockerignore` excluía esa carpeta del contexto de construcción. Se corrigió con el pull request #1 («fix: incluir docs en el contexto de Docker»), tras lo cual el CI pasó en verde y el despliegue en Render se completó correctamente.

6. Capturas de pantalla (evidencias)

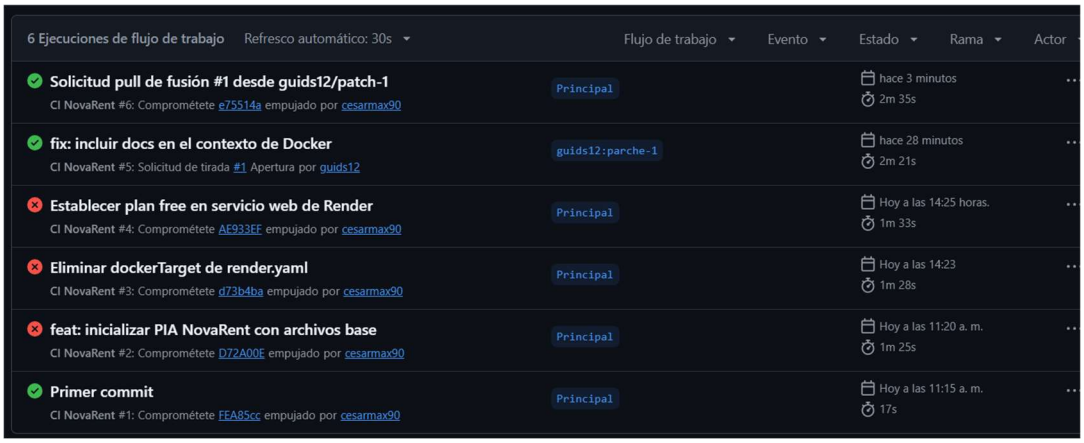


Figura 1. Historial de ejecuciones del workflow «CI NovaRent» en GitHub Actions: las ejecuciones #5 y #6 pasan en verde tras corregir el .dockerignore; las #2 a #4 fallaron.

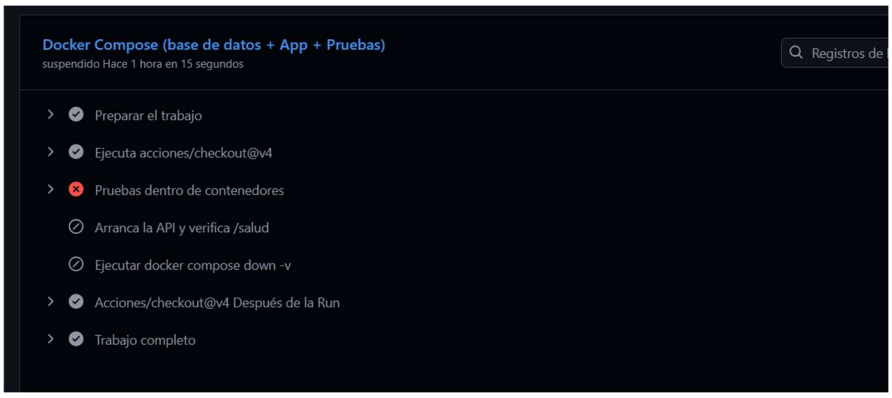


Figura 2. Fallo inicial del job «Docker Compose (base de datos + App + Pruebas)» en el paso «Pruebas dentro de contenedores», antes de la corrección.

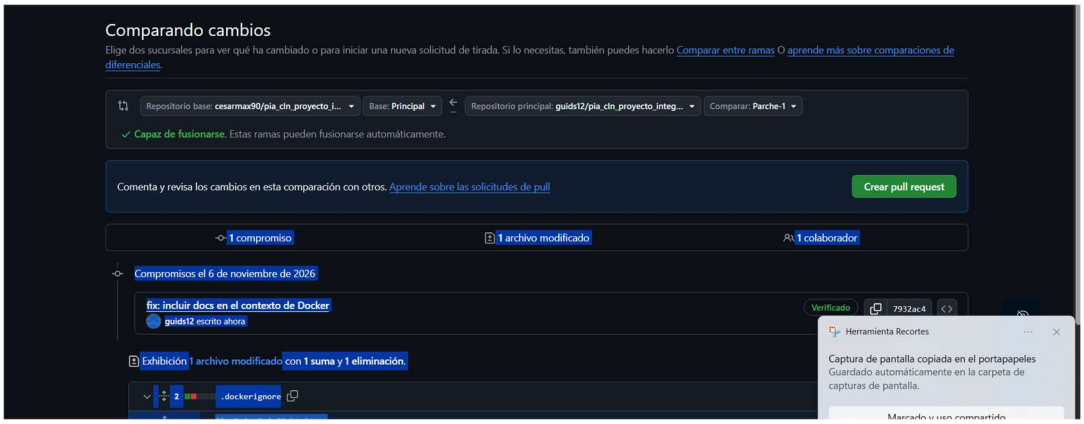


Figura 3. Pull request #1 de guids12 («fix: incluir docs en el contexto de Docker»): 1 commit y 1 archivo modificado (.dockerignore).

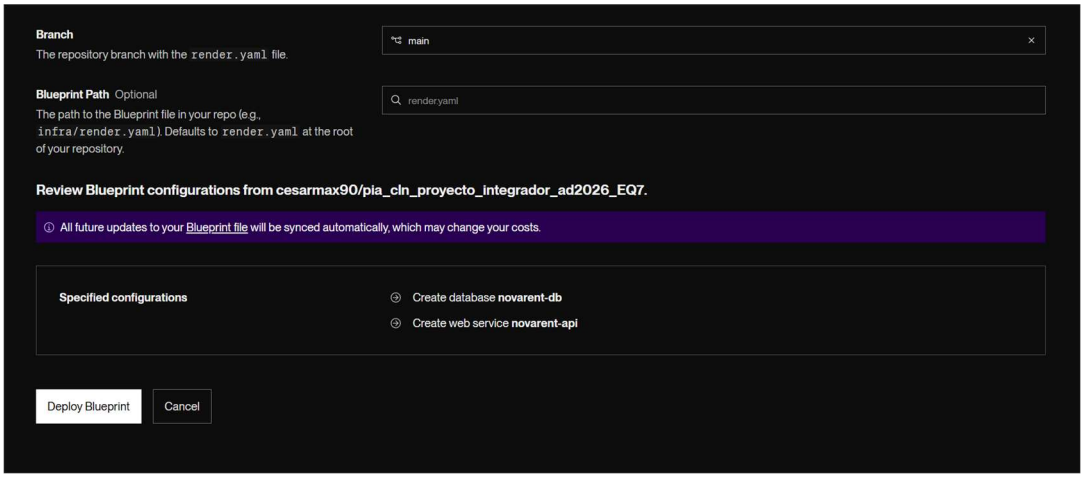


Figura 4. Blueprint de Render: creación de la base de datos novarent-db y del servicio web novarent-api a partir del repositorio del equipo.

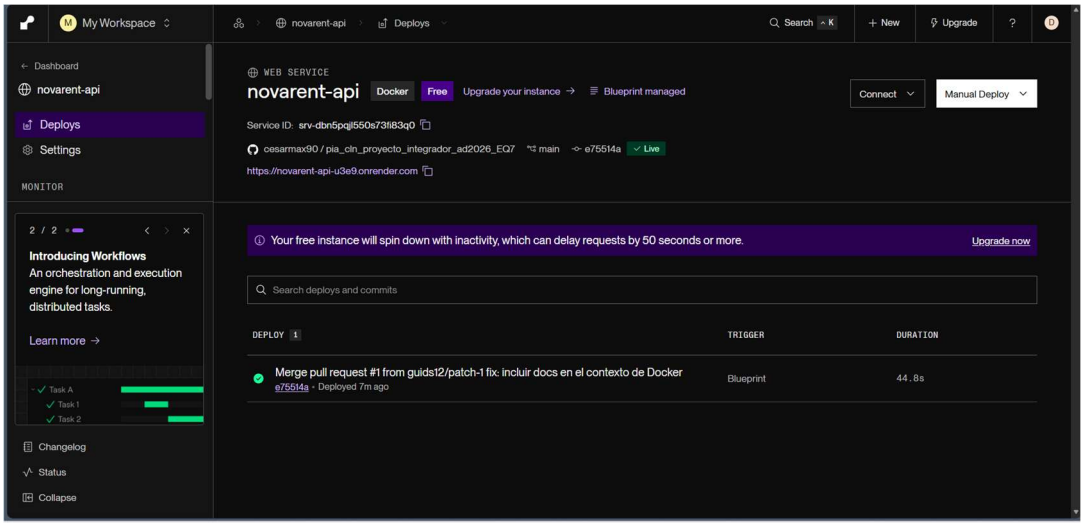


Figura 5. Servicio novarent-api en Render con estado Live, desplegado desde la rama main (commit e75514a) y con URL pública.



Figura 6. Respuesta del endpoint /salud de la aplicación desplegada: estado «ok», versión 1.0.0, entorno de producción y 2 sucursales cargadas desde la base de datos.



Figura 7. Documentación OpenAPI (Swagger) de la API NovaRent en /docs, con los grupos de endpoints de Catálogo y Reservaciones.

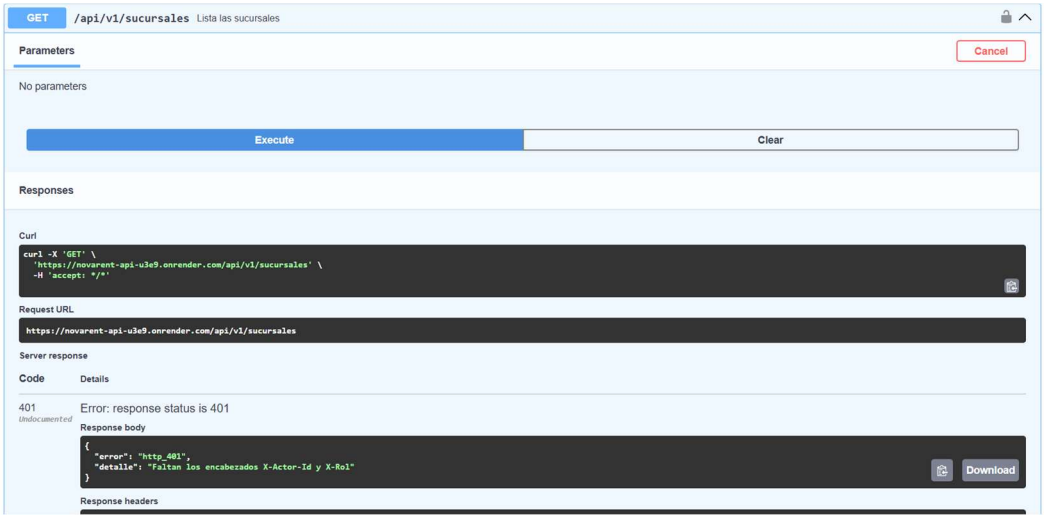


Figura 8. Respuesta 401 de GET /api/v1/sucursales sin encabezados de autenticación: la API exige X-Actor-Id y X-Rol.

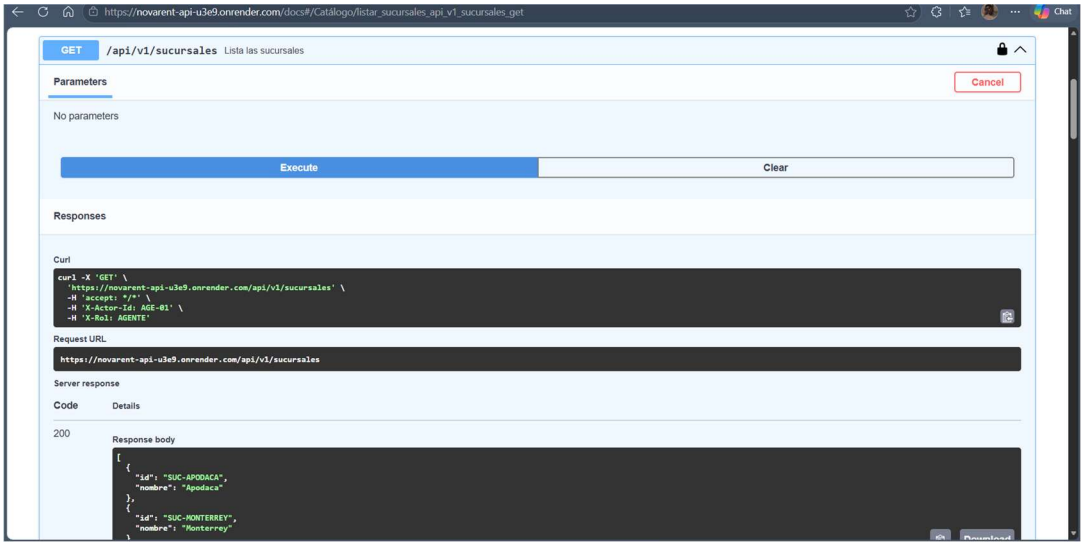


Figura 9. Respuesta 200 de GET /api/v1/sucursales en la aplicación desplegada, con X-Actor-Id: AGE-01 y X-Rol: AGENTE; devuelve las sucursales Apodaca y Monterrey.

7. Contribución por integrante

El historial de commits del repositorio refleja el trabajo de cada integrante:

Integrante	Área	Evidencia
Cesar Maximiliano Peña Mendoza (`cesarmax90`)	Repositorio, configuración de Docker/CI y despliegue en Render	Commits iniciales, ajustes de `render.yaml`, fusión del PR #1 y despliegue en Render
Diego Alberto Sierra Mendoza (`guids12`)	Diagnóstico y corrección del CI de Docker	Pull request #1: corrección de `.dockerignore`
Arturo Sebastian Pérez Zapata	Documentación	Evidencia de la presentación Eq7_NovaRent_PIA_Presentation.pdf

8. Registro de uso de inteligencia artificial

Se utilizó un asistente de IA (Claude, de Anthropic) como apoyo para comprender la guía de entrega, diagnosticar el fallo del CI y redactar este reporte. Las decisiones, la ejecución de los pasos en GitHub y Render y la verificación de los resultados fueron realizadas por el equipo.

Uso	Prompt (resumen)	Resultado	Revisión propia
Entender la guía de entrega	Se compartieron capturas de la guía del PIA y se preguntó qué hay que hacer y qué pasos seguir.	Lista de entregables, pasos y pendientes.	Se contrastó con la guía original del curso.
Revisar el estado del repositorio	Se compartió el enlace del repositorio y se pidió analizar qué ya estaba hecho.	Resumen del contenido y detección de que el job de Docker Compose fallaba.	Se verificó en la pestaña Actions de GitHub.
Diagnosticar el fallo del CI	Se pegó el log del paso «Pruebas dentro de contenedores».	Causa probable: `docs` excluida por `.dockerignore`.	Se aplicó el cambio y el CI pasó en verde.
Despliegue en Render	Capturas de cada pantalla del Blueprint y preguntas sobre qué hacer.	Indicaciones paso a paso (usar el repositorio de Cesar, revisar costos, verificar `/salud` y `/docs`).	Se comprobó el servicio en estado Live y las respuestas de la API.
Redacción del reporte	Se pidió generar el reporte final con lo necesario, en el formato del Word de la actividad.	Borrador de este documento.	Se completaron y corrigió información errónea e incompleta.
Comprobación de presentación	Se compartió un PDF sobre la presentación y se le pidió revisar posibles fallas,	Nos indico sugerencias y errores ortográficos.	Se corrigieron los errores ortográficos y ciertas sugerencias fueron tomadas en cuenta.

Este registro debe ajustarse a la política de uso de IA del curso.

9. Supuestos y limitaciones

Supuestos del BRD (Anexo C). Los valores que el BRD de la unidad 2 no fija están marcados como supuestos y se manejan como parámetros configurables: penalización por cancelación, tarifa por octavo de combustible, tarifa por hora de retraso y monto del depósito.

Limitaciones conocidas.

- Autenticación simulada mediante encabezados `X-Actor-Id` y `X-Rol` (ADR-007).
- Eventos de dominio en proceso, sin patrón Outbox (ADR-005).
- Sin migraciones de base de datos versionadas (ADR-004).
- El cargo por conductor joven no tiene monto definido en el BRD.
- El servicio desplegado usa el plan gratuito de Render, por lo que puede tardar en responder tras un periodo de inactividad.

10. Conclusiones

El proyecto cumple con la estructura mínima solicitada por el PIA: documentación de requerimientos y diseño, lógica de dominio con especificaciones, máquinas de estado y eventos, API REST con OpenAPI, persistencia con Repository y Unit of Work, pruebas con cobertura superior al mínimo, contenedores con Docker Compose, integración continua y despliegue público en Render.

Durante la entrega se detectó y corrigió un problema de empaquetado en Docker que impedía que el CI de contenedores pasara; su solución permitió validar por primera vez el proyecto con Docker real y completar el despliegue. Como trabajo futuro se identifican la autenticación real, el patrón Outbox para eventos y las migraciones versionadas.

El desarrollo e implementación de este proyecto representó una valiosa oportunidad para consolidar los conocimientos teóricos y prácticos de la ingeniería de software moderna, permitiéndonos comprender de primera mano la importancia del diseño orientado al dominio (DDD), la arquitectura en capas y la automatización mediante pipelines de CI/CD. A lo largo del proceso enfrentamos y resolvimos retos técnicos reales como la sincronización de contenedores y la gestión de dependencias en Docker, lo que fortaleció nuestra capacidad de trabajo en equipo, pensamiento crítico y resolución de problemas bajo estándares profesionales de calidad y pruebas de código.